

Question and Answer

Q1: Explain the term Programmable device.

Ans:

A **programmable device** is an electronic device whose functionality can be changed by loading a **program** or **configuration data**, allowing the same hardware to perform different tasks without any physical modification.

The information loaded into a programmable device depends on its architecture:

1. **Instruction-based programmable devices** (such as microcontrollers, microprocessors, computers, and PLCs) store a **program**, which is a sequence of instructions. The program is stored in memory (on-chip or off-chip), and the CPU fetches, decodes, and executes these instructions one by one. During execution, the **Arithmetic Logic Unit (ALU)** performs arithmetic operations (addition, subtraction, multiplication, etc.) and logical operations (AND, OR, NOT, XOR, etc.) as specified by the program.
2. **Hardware-configurable devices** (such as CPLDs and FPGAs) do not execute a sequence of instructions in the traditional sense. Instead, they receive a **configuration file (bitstream)** that programs the internal logic blocks and their interconnections. This process configures the hardware itself to implement the required digital circuit. Once configured, the hardware operates directly according to the implemented logic, rather than executing instructions sequentially.

Thus, the term **programmable device** refers to any electronic device whose operation or functionality can be modified by changing its stored program or hardware configuration.

Examples: Microcontrollers, Microprocessors, Computers, PLCs, DSP/DSC, CPLDs, and FPGAs.

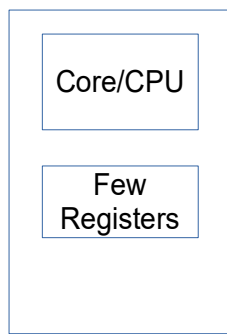
Q2: Explain the difference between Microprocessor and Microcontroller.

Ans:

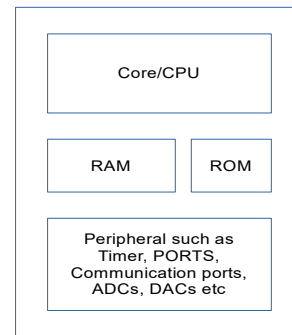
A **microprocessor** is a programmable integrated circuit that consists mainly of a **Central Processing Unit (CPU)**. It requires external memory (RAM/ROM) and peripheral devices (I/O ports, timers, communication interfaces, etc.) to build a complete computing system.

A **microcontroller** is a programmable integrated circuit that integrates the **CPU, memory, input/output ports, timers, communication interfaces, and other peripherals on a single chip**. It is designed for dedicated embedded applications.

With a diagram, the above point can be put as in following,



The Microprocessor



The Microcontroller

Take an example of temperature controlling Embedded application/System. There speed of FAN is to be controlled as per the temperature measured by controlling the output of DC DC converter with a PWM signal. The measured temperature needs to be updated on 16x2 LCD.

Comparison with two different design (Microprocessor versus Microcontroller based design):

Microprocessor based design (Say Intel 8085)	Microcontroller based design (Say c8051f120)
The program memory and RAM memory needs to be interfaced as they are not available onchip.	The program can be stored in onchip program memory.
The microprocessor has bus available. Now this bus has both 'ADDRESS' and 'DATA' on it at different time. So required data, must be separated, latched and be hold. Hence to latch up the data required to drive PWM, to take input of Digital data converted by ADC, to send data to LCD, one need to include several Ics.	This will not be required in Microcontroller as it has Ports and Port pin available.
Analog to Digital (ADC) needs to be interfaced.	Some microcontroller (e.g c8051f120) has onchip ADC. So extra ADC IC is not required.
Due to above three points, the overall size of circuit and PCB is more. The complexity and cost is hence more, depending on the application.	Onchip memory and peripheral makes it easy and cost effective.

Q3: Discuss the difference between how execution carried out in a Computer and in a Microcontroller.

Ans:

Both a **computer** and a **microcontroller** execute programs using the **Fetch–Decode–Execute cycle**. In both cases, the CPU fetches instructions from memory, decodes them, and executes them sequentially. However, they differ in the way the program is stored, loaded, and used.

Computer	Microcontroller
The program is usually stored on a secondary storage device such as an SSD or hard disk.	The program is permanently stored in the on-chip Flash/ROM memory.
When the computer is powered on, the operating system is loaded into RAM before user applications are executed.	When powered on, the microcontroller starts executing the user program directly from the on-chip program memory. The program memory has sequence of Opcode and Operands.
The operating system manages memory, files, input/output devices, and multiple running programs.	Usually executes a single dedicated application without requiring an operating system (although an RTOS may be used in some applications).
Multiple programs can run simultaneously under the control of the operating system.	Typically executes one application continuously to perform a specific task.
Mainly designed for general-purpose computing.	Designed for dedicated embedded control applications.
Examples: Desktop computers, laptops, servers.	Examples: Washing machines, microwave ovens, digital energy meters, traffic light controllers, automotive ECUs.

Execution Process

Computer

1. The computer is powered on.
2. The BIOS/UEFI initializes the hardware.
3. The operating system is loaded from secondary storage into RAM.
4. The operating system starts running.
5. When a user launches an application, its executable code is loaded into RAM.
6. The CPU fetches, decodes, and executes the application's instructions.

Microcontroller

1. The microcontroller is powered on or reset.
2. The CPU begins execution from a predefined reset address in the on-chip program memory.
3. The user program is fetched, decoded, and executed.
4. The program continuously interacts with sensors, actuators, communication interfaces, and other peripherals to perform the intended control task.

Although both computers and microcontrollers execute instructions using the same **Fetch–Decode–Execute** mechanism, a **computer** relies on an operating system to manage multiple applications and hardware resources, whereas a **microcontroller** is designed to execute a dedicated embedded

program directly from its program memory, providing a compact, low-cost, and efficient solution for control applications.

Q4: Give details of some parameters affecting the speed of code execution for a microcontroller.

Ans:

The execution speed of a microcontroller primarily depends on its **clock frequency**. A higher clock frequency generally enables the microcontroller to execute more operations per second. However, two microcontrollers operating at the **same clock frequency** may exhibit significantly different execution speeds due to differences in their architecture, instruction execution mechanism, and hardware resources.

The major factors affecting the execution speed are discussed below.

1. Clock Frequency

The clock provides the timing signal for the CPU. The execution speed generally increases with an increase in clock frequency.

For example, a microcontroller operating at **100 MHz** can potentially execute instructions faster than one operating at **20 MHz**, provided both require the same number of clock cycles per instruction.

2. Machine Cycle and Instruction Execution

Every instruction requires one or more **machine cycles** (clock cycles) for execution.

A microcontroller capable of executing most instructions in a **single clock cycle** will complete a program faster than another microcontroller that requires multiple clock cycles for executing the same instructions.

Many modern microcontrollers provide **single-cycle instruction execution**, allowing more instructions to be executed per second. This not only improves execution speed but also enables the execution time of a program to be estimated more accurately (deterministic execution).

3. Processor Architecture

The internal architecture of a microcontroller has a significant influence on its execution speed.

- **Word Length (Data Path):** A processor with a wider data path (such as 32-bit or 64-bit) can process larger data in a single instruction. For example, a 32-bit microcontroller can perform 32-bit integer arithmetic in one operation, whereas an 8-bit microcontroller may require multiple instructions to perform the same operation. This reduces the execution time of arithmetic and logical operations.
- **Dedicated Arithmetic Hardware:** Many modern microcontrollers include dedicated hardware units such as a **hardware multiplier**, **Multiply-Accumulate (MAC) Unit**, **Floating Point Unit (FPU)**, and **Trigonometric Math Unit (TMU)**. These units significantly accelerate mathematical computations. For example, a microcontroller equipped with an **FPU** (e.g., ARM Cortex-M4F or Cortex-M7F) performs floating-point

arithmetic much faster than a microcontroller that implements floating-point operations entirely through software.

- **Pipeline Architecture:** Many processors employ instruction pipelining, in which different stages of multiple instructions are executed simultaneously. This increases instruction throughput and improves overall execution speed.
- **Cache memory:** It reduces the flash memory access time.

4. Instruction Set Architecture (ISA)

The design of the instruction set also affects execution speed.

- **RISC (Reduced Instruction Set Computer)** architecture uses a smaller number of simple instructions, many of which can be executed in a single clock cycle. This results in faster instruction execution.
- **CISC (Complex Instruction Set Computer)** architecture provides more complex instructions capable of performing multiple operations. Although this reduces the number of instructions in a program, individual instructions may require multiple clock cycles for execution.

5. Memory Architecture

The organization of program and data memory also influences execution speed.

Microcontrollers based on the **Harvard Architecture** have separate memory spaces and buses for program and data. This allows simultaneous access to instructions and data, thereby improving execution speed.

In contrast, **Von Neumann Architecture** uses a single memory space and bus for both program and data, which may reduce performance because instruction fetching and data transfer cannot occur simultaneously.

6. Compiler Optimisation

The efficiency of the generated machine code also affects execution speed.

An optimising compiler reduces unnecessary instructions, efficiently utilizes processor registers, and generates optimised machine code, thereby improving the overall execution speed of the program.

The execution speed of a microcontroller is determined not only by its **clock frequency** but also by factors such as **machine cycle duration, processor architecture, word length, dedicated arithmetic hardware (Multiplier, MAC, FPU, TMU), instruction set architecture (RISC/CISC), memory organisation, and compiler optimisation**. Therefore, two microcontrollers operating at the same clock frequency can exhibit significantly different execution performance depending on their internal architecture and hardware features.

Q5: Explain hoe pipilining and Cache memory reduces speed of execution.

Ans:

Pipelining and **cache memory (or Flash prefetch/cache)** are completely different concepts, although **both improve execution speed**. They are often confused with each other, as both reduce the effective execution time of instructions.

Here's a clear distinction.

Pipelining	Cache / Flash Wait-State Reduction
Improves the CPU's execution process .	Improves memory access speed .
Overlaps the execution of multiple instructions.	Stores frequently accessed instructions/data in a faster memory.
Works inside the CPU core .	Works between the CPU and Flash memory .
Increases instruction throughput.	Reduces the time required to fetch instructions from Flash.
Does not reduce Flash access time.	Does not overlap instruction execution.

1. Pipelining

Without pipelining, the CPU executes instructions one after another.

```
Instruction 1 : Fetch → Decode → Execute
Instruction 2 : Fetch → Decode → Execute
Instruction 3 : Fetch → Decode → Execute
```

Suppose each stage takes one clock cycle.

Cycle Operation

```
1    Fetch I1
2    Decode I1
3    Execute I1
4    Fetch I2
5    Decode I2
6    Execute I2
```

Total = **6 cycles for 2 instructions**.

With a 3-stage pipeline,

Cycle Stage 1 Stage 2 Stage 3

```
1    Fetch I1
2    Fetch I2 Decode I1
3    Fetch I3 Decode I2 Execute I1
4    Fetch I4 Decode I3 Execute I2
```

Now one instruction completes every clock cycle after the pipeline is filled.

Pipelining increases throughput.

2. Flash Wait States and Cache

Suppose your CPU runs at **200 MHz**.

One clock cycle = **5 ns**.

But suppose internal Flash memory needs **20 ns** to return an instruction.

Without cache,

```
CPU asks Flash for instruction.  
CPU waits...  
CPU waits...  
CPU waits...  
Instruction arrives.
```

That means

$20\text{ ns} / 5\text{ ns} = 4\text{ clock cycles}$

The CPU wastes several clock cycles simply waiting for the instruction.

To overcome this,

Modern microcontrollers use

- Instruction Cache
- Prefetch Buffer
- Branch Target Buffer
- Flash Accelerator

Now frequently used instructions are already stored in fast SRAM-like memory.

Instead of

CPU → Flash (20 ns)

the access becomes

CPU → Cache (1 cycle)

Thus the waiting time is greatly reduced.

Analogy

Imagine a chef cooking.

Pipelining

One chef performs

1. Wash vegetables
2. Cut vegetables
3. Cook

Without pipelining

Wash A
Cut A
Cook A
Wash B
Cut B
Cook B

With pipelining

Wash B
Cut A
Cook A

Many tasks happen simultaneously.

Cache

Now suppose vegetables are stored in another building.

Every time the chef needs vegetables,
he walks there.

That is Flash memory.

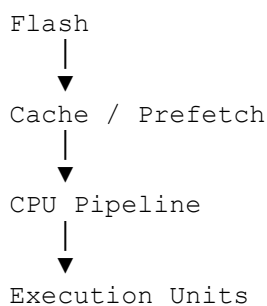
Instead, keep today's vegetables beside the stove.

That is cache.

The chef no longer wastes time walking.

Relationship

The two techniques actually **complement** each other.



- **Cache** ensures the pipeline receives instructions quickly.
- **Pipeline** ensures the CPU executes those instructions efficiently.

If there is **no cache**, the pipeline may **stall** because it has no instruction to execute.

If there is **no pipeline**, cache still speeds up instruction fetch, but the CPU executes instructions one after another.

Q6: Explain Von numen Architecture and their applications.

Ans:

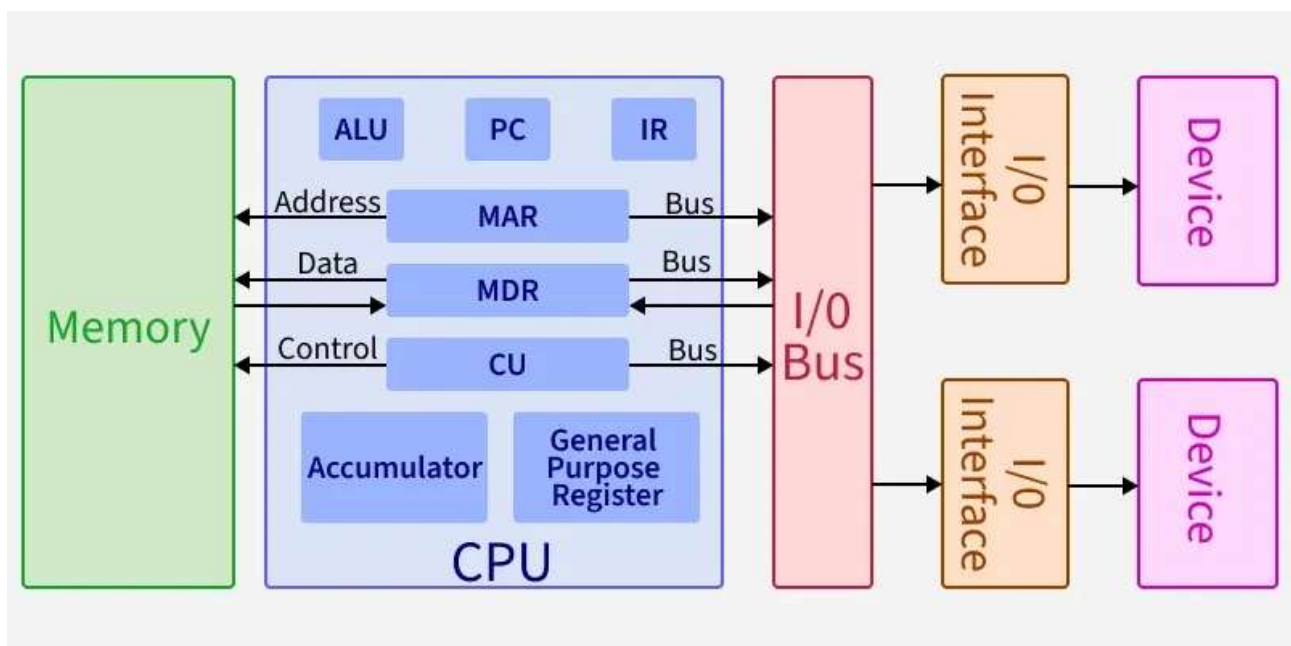
Von Neumann architecture is a computer design where instructions and data are stored in the same memory space. This means the CPU fetches both instructions and data from the same memory, using the same pathways.

The structure described in the figure outlines the basic components of a computer system, particularly focusing on the memory and processor. It is made up with three main components:

- CPU
- Memory
- I/O Devices

We have these 3 components lets us see more about them in detail:

The central processing unit (CPU) is the main part of a computer that controls how it works. It is made up of the control unit, the arithmetic logic unit (ALU) and Registers (not RAM). The CPU handles instructions from programs, processes data, stores information, and produces results. Without the CPU, a computer cannot perform tasks or run any applications.



CU (Control Unit)

The control unit manages how the processor works by sending control signals. It decides how data should move inside the computer, controls input and output operations, and fetches the instructions from memory for execution.

ALU (Arithmetic and Logic Unit)

The arithmetic and logic unit is the part of the CPU that handles the calculations and decision-making tasks. It performs arithmetic operations like addition and subtraction, logical operations such as comparisons, and tasks like shifting bits in data.

Registers

Registers are the fastest type of memory located inside the CPU. They temporarily store information that the processor is currently working on, making program execution and operations faster and more efficient. Register serve as the CPU's primary working memory.

- **PC (Program Counter):** Keeps track of the address of the next instruction to be executed.
- **IR (Instruction Register):** Holds the current instruction being executed.
- **MAR (Memory Address Register):** Stores the address of the memory location being accessed.
- **MDR (Memory Data Register):** Temporarily holds data being transferred to or from memory.
- **Accumulator:** A register that stores intermediate results of arithmetic and logic operations.
- **General Purpose Registers:** Used for temporary storage of data during processing.

Bus

The bus is a communication system that transfers data, addresses, and control signals between the CPU, memory, and I/O devices. In Von Neumann architecture, a single bus is shared for both data and instructions, which can create a bottleneck (known as the Von Neumann bottleneck).

I/O Bus

- **I/O Interface:** Connects the CPU and memory to input/output devices.
- **Device:** Refers to external hardware like keyboards, monitors, or storage devices.

Key Characteristics

1. **Single Memory for Data and Instructions:** Both data and program instructions are stored in the same memory.
2. **Shared Bus:** A single bus is used for transferring data, addresses, and control signals, which can limit performance.
3. **Sequential Execution:** Instructions are executed one at a time in a sequential manner.

Von Neumann bottleneck

Whatever we do to enhance performance, we cannot get away from the fact that instructions can only be done one at a time and can only be carried out sequentially. Both of these factors hold back the competence of the CPU. This is commonly referred to as the 'Von Neumann bottleneck'. We can provide a Von Neumann processor with more cache, more RAM, or faster components but if original gains are to be made in CPU performance then an influential inspection needs to take place of CPU configuration.

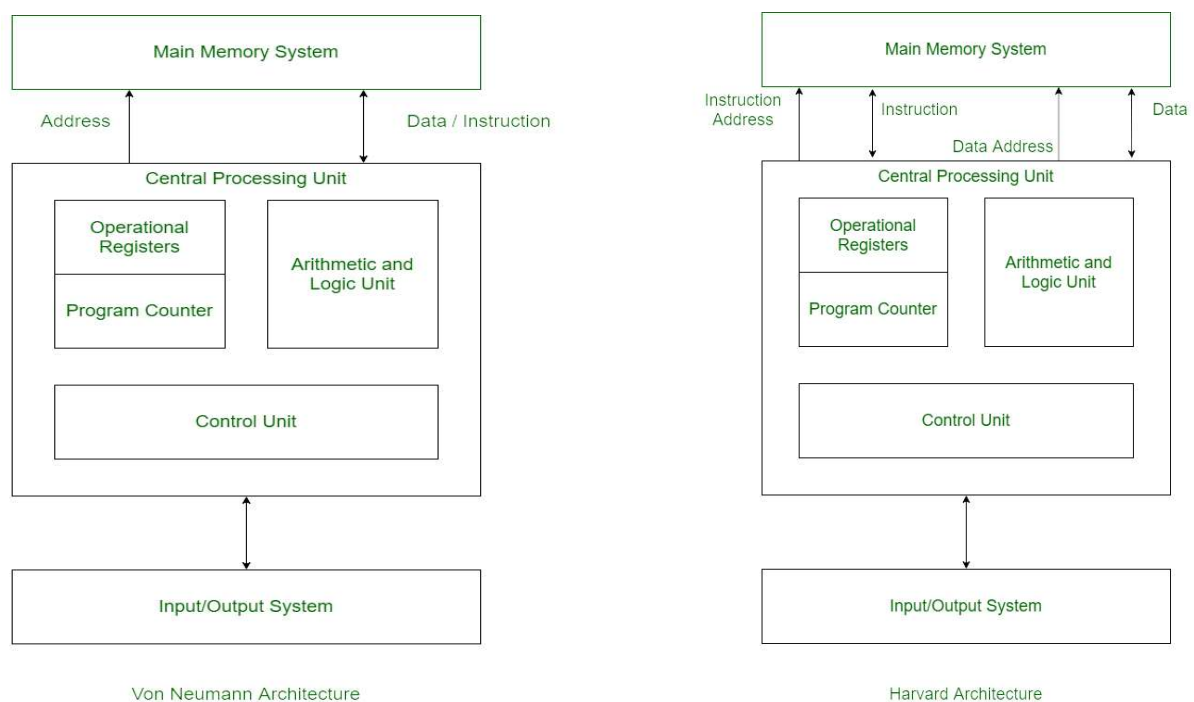
Applications

Von Neumann architecture is the foundation of most modern computing systems, where both instructions and data are stored in the same memory. Here are the some applications:

- **Personal Computers and Laptops:** Most PCs and laptops (Intel/AMD) use a shared memory that stores both programs and data, enabling efficient running of software like Windows and browsers. **Example:** Intel Core i7, AMD Ryzen processors.

- **Smartphones and Tablets:** ARM-based devices use the shared memory model to support multitasking and power efficiency in apps on iOS and Android. **Example:** Qualcomm Snapdragon, Apple A-series chips
- **Embedded Systems:** Microcontrollers in cars, IoT devices, and appliances use Von Neumann architecture for simple, cost-effective processing of instructions and data in the same memory. **Example** Arduino Uno (ATmega328), older ARM Cortex-M designs.
- **Servers and Cloud Computing:** Servers utilize shared memory systems to handle large-scale software and data tasks for services like AWS and Netflix.
- **Gaming Consoles:** Consoles like PS5 and Xbox use unified memory to run game code and manage inputs, enabling smooth gameplay.

Q7: Explain difference between Von Neumann and Harvard Architecture, with required diagram.



Von Neumann:

In this architecture, **program instructions and data are stored in the same memory** and share a **single communication bus** between the CPU and memory.

Working

1. The CPU fetches an instruction from memory.
2. The instruction is decoded.
3. If the instruction requires data, the CPU again accesses the same memory.
4. The result is written back to memory.

Since both instruction fetching and data transfer use the **same bus**, they **cannot occur simultaneously**.

Advantages

- Simple hardware design.
- Lower manufacturing cost.
- Flexible memory allocation.
- Easier to design and program.

Disadvantages

- Program and data compete for the same memory bus.
- Instruction fetch and data access cannot occur simultaneously.
- This limitation is known as the **Von Neumann Bottleneck**.
- Lower execution speed compared to Harvard architecture.

Applications

- Personal computers
- Laptops
- Desktop computers
- Older microprocessors such as Intel 8086

2. Harvard Architecture

The **Harvard Architecture** was originally developed for the **Harvard Mark I** computer.

In this architecture, **program memory** and **data memory** are physically separate and communicate with the CPU through **independent buses**.

Working

1. The CPU fetches the next instruction from the program memory.
2. Simultaneously, it can read or write data from the data memory.
3. Since separate buses are used, both operations occur in parallel.

This increases the instruction throughput and improves overall performance.

Advantages

- Simultaneous instruction fetch and data access.
- Higher execution speed.
- Better processor performance.
- Suitable for real-time and embedded systems.
- Supports efficient pipelining.

Disadvantages

- More complex hardware.
- Higher manufacturing cost.

- Separate memory organization increases design complexity.
- Memory cannot be shared dynamically between program and data.

Applications

- Most modern microcontrollers (PIC, AVR, 8051 variants, ARM Cortex-M)
- Digital Signal Processors (DSPs)
- Embedded systems

Comparison between Von Neumann and Harvard Architectures

Feature	Von Neumann Architecture	Harvard Architecture
Memory	Single memory for program and data	Separate program and data memories
Bus	Single bus	Separate buses
Instruction Fetch and Data Access	Cannot occur simultaneously	Can occur simultaneously
Execution Speed	Lower	Higher
Hardware Complexity	Simple	More complex
Cost	Lower	Higher
Performance	Limited by Von Neumann Bottleneck	Higher throughput
Typical Applications	Computers, general-purpose processors	Microcontrollers, DSPs, embedded systems

Von Neumann Bottleneck

In a Von Neumann architecture, the CPU uses the same bus for both instruction fetch and data transfer.

For example:

```
Time t1 : Fetch Instruction
Time t2 : Read Data
Time t3 : Execute
Time t4 : Write Result
```

The CPU must wait for one memory operation to complete before starting another, reducing execution speed.

Harvard Architecture Advantage

In Harvard architecture:

```
Instruction Fetch  ──────────▶ Program Memory
Data Read/Write   ──────────▶ Data Memory
```

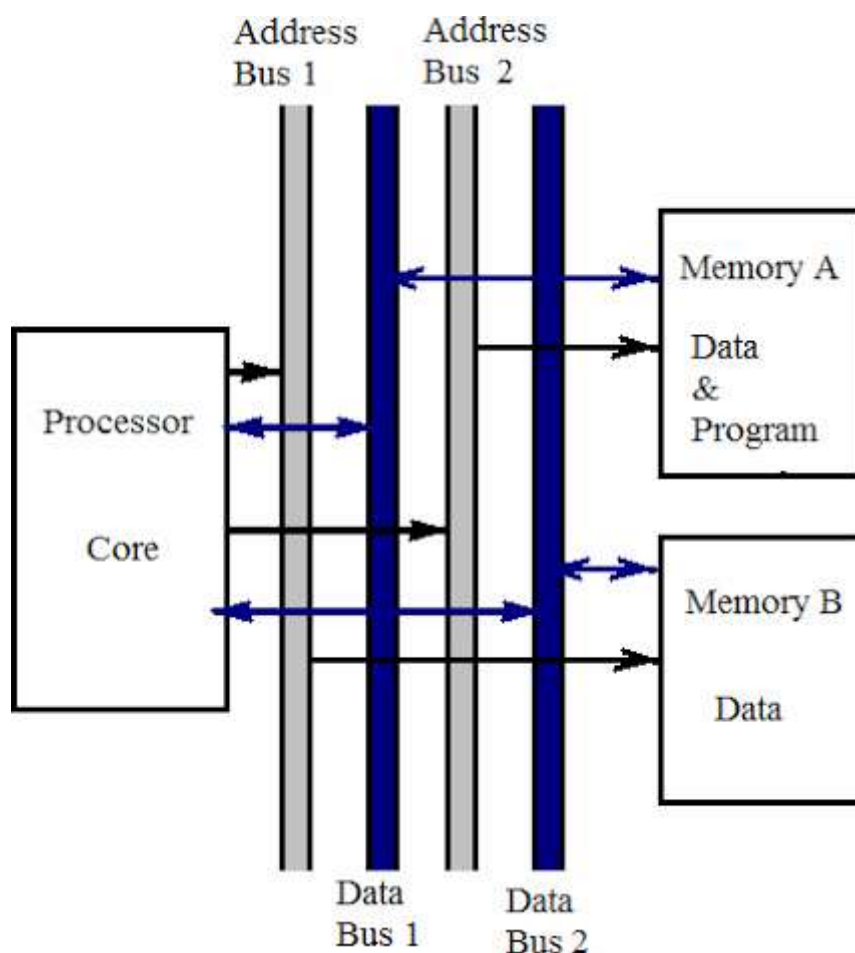
Both operations occur simultaneously, reducing idle CPU time and improving performance.

Conclusion

The **Von Neumann Architecture** uses a **single memory and a single bus** for both instructions and data, making it simple and economical but limiting execution speed due to the **Von Neumann bottleneck**.

The **Harvard Architecture** uses **separate memories and buses** for instructions and data, allowing simultaneous instruction fetch and data access. This results in higher execution speed and makes Harvard architecture the preferred choice for **modern microcontrollers, digital signal processors, and real-time embedded systems**.

The following figure is for question on next page.....



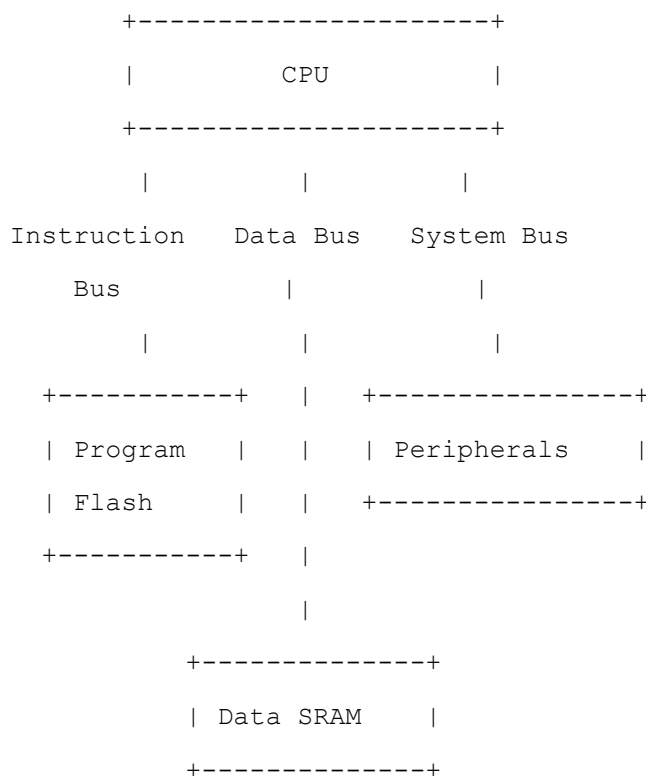
Q8: Explain Modified Harvard architecture.

The **Modified Harvard Architecture** is a computer architecture that combines the features of both the **Von Neumann Architecture** and the **Harvard Architecture**.

Like the Harvard architecture, it uses **separate buses** for instruction and data transfer, allowing simultaneous instruction fetching and data access. However, like the Von Neumann architecture, it provides a **unified memory address space**, making programming easier and more flexible.

As a result, the Modified Harvard Architecture achieves higher execution speed while simplifying software development.

Block Diagram



Note: The exact number of buses depends on the processor. For example, ARM Cortex-M processors commonly have separate instruction, data, and system buses based on the AMBA bus architecture.

Also refer diagram given on previous page.

Working

The operation of the Modified Harvard Architecture is as follows:

1. The CPU fetches instructions from the **program memory (Flash)** through the **instruction bus**.

2. Simultaneously, it reads from or writes to the **data memory (SRAM)** through the **data bus**.
 3. The system bus is used to communicate with peripherals such as timers, ADCs, UARTs, GPIOs, and other devices.
 4. Since instruction fetch and data access occur simultaneously, the processor executes programs faster than a conventional Von Neumann architecture.
-

Features

- Separate instruction and data buses. A more accurate statement is:
Modified Harvard Architecture provides separate instruction and data paths (or buses), while presenting a unified memory address space to the programmer. Depending on the implementation, the instruction and data may reside in physically separate memories (e.g., Flash and SRAM in a microcontroller) or in a common main memory with separate instruction and data caches (e.g., Intel Core and ARM Cortex-A processors).
 -
 - Simultaneous instruction fetch and data transfer.
 - Unified memory address space for easier programming.
 - Higher instruction throughput.
 - Better support for pipelining and cache memory.
 - Improved overall execution speed.
-

Advantages

- Faster execution than the Von Neumann architecture.
 - Efficient utilization of CPU resources.
 - Supports pipelining and cache memory.
 - Easier software development because the programmer sees a single memory map.
 - Suitable for high-performance embedded systems.
-

Disadvantages

- More complex hardware design.
 - Requires additional buses and control logic.
 - Slightly higher implementation cost than a simple Von Neumann architecture.
-

Applications

The Modified Harvard Architecture is widely used in modern processors and microcontrollers, including:

- ARM Cortex-M series
- ARM Cortex-A series
- Intel Core processors

- AMD Ryzen processors
- Many Digital Signal Processors (DSPs)

Comparison

Feature	Harvard	Modified Harvard
Program and Data Memory	Separate	Usually separate (physically or logically)
Program and Data Buses	Separate	Separate
Unified Address Space	No	Yes
Simultaneous Instruction Fetch and Data Access	Yes	Yes
Programming Simplicity	Moderate	High

Example: ARM Cortex-M

Consider an ARM Cortex-M microcontroller.

Flash Memory → Stores program instructions

SRAM → Stores variables and stack

Peripherals → GPIO, ADC, UART, Timers

During execution:

- The CPU fetches an instruction from **Flash** via the **instruction bus**.
- At the same time, it reads or writes variables in **SRAM** via the **data bus**.
- Simultaneously, it may access a peripheral (e.g., a UART register) via the **system bus**.

These parallel operations improve execution speed and are one of the reasons ARM Cortex-M microcontrollers offer high performance even at moderate clock frequencies.

Conclusion

The **Modified Harvard Architecture** combines the high performance of the Harvard architecture with the programming convenience of the Von Neumann architecture. By providing separate instruction and data buses while maintaining a unified memory address space, it allows simultaneous instruction fetch and data access, leading to faster execution. This architecture is therefore the preferred choice for most modern microcontrollers and processors.

Q9: Classify the microcontroller 8051 and ARM in to Von numen and/or Harvard architecture with required justification and reason for the classification answered.

1. 8051 Microcontroller

The **8051** is a **Harvard Architecture** microcontroller.

Reason

The 8051 has **separate address spaces** for:

- **Program Memory (Code Memory)**
- **Data Memory (RAM)**

Although the external buses are multiplexed to reduce pin count, internally the CPU treats program memory and data memory separately.

For example,

- **MOVC** instruction reads **code memory**.
- **MOV** instruction reads **data memory**.

The CPU can fetch instructions from program memory while accessing data memory, which is the characteristic feature of Harvard architecture.

Memory Organization

Code Memory (64 KB)



Data Memory (64 KB)

Notice that the **address spaces are separate**.

Therefore,

8051 is a Harvard architecture microcontroller.

2. ARM Processor

The answer depends on **which ARM** you are referring to.

ARM Cortex-A (Application Processors)

Examples:

- Cortex-A7

- Cortex-A53
- Cortex-A72

These processors run Linux, Android, Windows, etc.

They use a

Modified Harvard Architecture

Reason:

Internally,

- Separate Instruction Cache
- Separate Data Cache

But

- Single unified memory address space

Therefore instruction and data can be accessed simultaneously while programmers see only one memory map.

ARM Cortex-M (Microcontrollers)

Examples

- Cortex-M0
- Cortex-M3
- Cortex-M4
- Cortex-M7
- STM32
- LPC
- TM4C
- Tiva C

These also use

Modified Harvard Architecture

Reason:

Internally they have

- Instruction Bus
- Data Bus
- System Bus

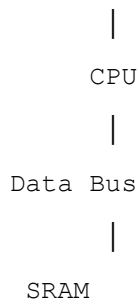
These buses operate simultaneously.

For example

Flash Memory

|

Instruction Bus



The programmer, however, sees

```

0x00000000  Flash
0x20000000  SRAM
0x40000000  Peripherals
  
```

Everything exists in one unified address map.

Hence ARM Cortex-M is **not a pure Harvard architecture**, but a **Modified Harvard Architecture**.

Comparison

Processor	Classification	Reason
8051	Harvard	Separate code and data memory spaces
AVR (ATmega328P)	Harvard	Separate program Flash and data SRAM
PIC16/PIC18	Harvard	Separate program and data memories
ARM Cortex-M	Modified Harvard	Separate instruction/data buses but unified address space
ARM Cortex-A	Modified Harvard	Separate I-cache and D-cache with unified memory
Intel 8086	Von Neumann	Single memory space and bus
Intel Core i7	Modified Harvard	Separate caches but unified memory

A point often misunderstood, is ..

Many books say

"ARM is Harvard."

Others say

"ARM is Von Neumann."

Both are **incomplete**.

The correct statement is

Modern ARM processors implement a Modified Harvard Architecture.

They have

- Separate instruction and data paths (Harvard feature)

- Unified memory address space (Von Neumann feature)

This combines the advantages of both architectures.

Architecture	Examples
Von Neumann	Intel 8086, early processors
Harvard	8051, AVR, PIC, DSPs
Modified Harvard	ARM Cortex-M, ARM Cortex-A, Intel Core processors

Q10: List and explain the criteria for selection of microcontroller, for implementation of desired application/system.

The following points should be considered,

1. Speed (clock frequency, machine cycle, MAC, TMU etc).
2. Memory (RAM and Program memory).
3. Peripherals (ADC, DAC, Ports, Communication protocols, Timer, PWM, Capture compare etc).
4. Power consumption and Power down mode.
5. Pin counts.
6. Budget available and Free availability of IDE, Cost of evaluation board and Hardware programmers, Information and support provided.

Speed depends on various factor which is well discussed in other question and answer.

The memory is important in following scenario mainly:

1. When the Operating system needs to be flashed rather than stand alone embedded program.
2. When program includes heavy mathematical computation which needs to store many floating point numbers.

Peripherals available onchip should be compared for their faures. For example the maximum sample rate for ADC in c8051f120 is 500 KSPS. The application requiring more sampling speed, requires either to use external ADC chip or some other microcontroller.

Some times, only few timers are available and one of the Timer is used by Operating system (in RTOS based code). Hence the numbers of timers are also important.

Similar to above two example, one need to check all details of required internal peripherals and it should meet the requirement.

Power consumption and pin count can be important in some special application. When an embedded system needs to be accommodated in small space, pin counts can be important. When application needs to be run with small battery and expected duration of operation is more, the power consumption has to be reduced and hence this choice is important.

Many a times, the development team is assign a small budget for their research and development activities. The cost of board and development accessories are important in that case.

Q11: Explain various Peripherals available in microcontroller.

Ans:

The following peripherals are available,

1. General Purpose Input output Ports and Port Pins:

This is most common requirement. Total port pin should be greater than requirement.

The port pin can be open drain or push pull pin. Open drain pins are useful specially when one need to drive a 5 Volt (or similar) IC from 3.3 V microcontroller. Push pull pin do not require pull up resistor.

The Sink and source current are important feature and if not sufficient, an external Buffer IC or transistor pair can be used to increase the capacity.

Cross bar facility can give some flexibility to map the physical port pin to internal function of microcontroller.

2. Timer and Counter:

Timer is used to provide required time delay precisely. The execution of interrupt service routine during a soft delay (provided by say for loop), can be severely affected, where as timer goes on its counting even during service of ISR, hence it is more accurate. The timer gets overflow (or underflow) after certain counts and flags it by a flag bit or geneates the interrupt. The desired action can be taken after overflow of timer.

Counter on the other hand counts the number of external pulses (which may occur at irregular interval). Hence the action require to execute at some counts can be taken care by counter.

The preciseness of smallest delay and highest possible delay depends on clock frequency and maximum count (timer bits) of the timer.

3. Communication Protocols:

The communication can be done by Ports and Serial Communication Pins.

Serical communication requires to stream and receive data in a format and follows some pattern and rules, called protocols.

These serial protocols are developed for some speed range and distance of communication. Various comminication supported by microcontrollers are UART, USART, USB, SPI, I2C, SMBUS etc.

4. Watch Dog:

A watch dog is a timer, which can reset the microcontroller if it is allowed to overflow. In complex embedded system, there are changes situation in which code execution gets hanged. There is a special routine which resets the watch dog at some regular interval. This interval is less than the overflow time of watch dog and hence in normal situation it sucessfuuly resets the watchdog before it overflows. In faulty/hang situation, that special routine resetting the watchdog also gets hanged, and hence watchdog timer will reache to its maximum count and overflows. This resets the microcontroller which help it to come out from hang situation.

5. ADC and DAC: